

인공지능 더 안전하게 활용하기

지금까지 언어 모델에 기반한 인공지능 프로그램을 개발하는 기술에 대해 배웠습니다. 하지만 이 기술을 어떻게 사용하느냐에 따라 우리를 돕는 도구가 될 수도 있고 해를 끼칠 위험 요소가 될 수도 있습니다. 이번 장에서는 인공지능에 기반한 프로그램을 개발할 때 주의해야 할 요소를 알아보고 로컬에서 언어 모델을 활용하는 방법도 살펴보겠습니다



16-1 로컬에서 라마와 임베딩 모델 사용하기

16-2 LLM에 기반한 서비스 발전시키기



16-1 로컬에서 라마와 임베딩 모델 사용하기

대규모 언어 모델의 API를 활용하면 해당 기업의 서버에서 계산을 처리하고 결과만 받아 개인 컴퓨터에서 실행됩니다. 그래서 고성능 컴퓨터가 없어도 인터넷만 연결되면 어디서든 대규모 언어 모델을 활용할 수 있습니다. 하지만 API를 활용하는 방식이 적합하지 않은 상황도 있습니다. 인터넷이 연결되지 않는 환경에서도 실행되는 언어 모델 기반 프로그램을 개발해야 할 수도 있고, API 호출 비용이 부담스러울 수도 있습니다. 또한 RAG 기반 프로그램을 개발할 때 회사의 기밀 문서를 다룰 경우 문제가 발생할 수도 있습니다. 오픈AI의 임베딩 모델을 사용하면 해당 문서가 인터넷을 통해 오픈AI의 서버로 전송됩니다. 비록 오픈AI가 사용자의 데이터를 사용하지 않는다고 하더라도 회사의 보안 규정상 자료를 외부로 전송하는 것을 금지하고 있다면 그에 따른 문제가 발생할 수 있습니다. 이런 경우 로컬 환경에서 언어 모델을 구동할 수 있다면 좋겠죠.

API를 사용하지 않고 로컬 컴퓨터에서 챗봇이나 RAG 기반 프로그램을 개발하는 방법을 알아 보겠습니다. 여기서는 소규모 언어 모델로 자주 활용되는 메타의 라마 모델을 사용합니다.

Do it! 실습 메타의 라마 모델을 로컬에서 구동하기

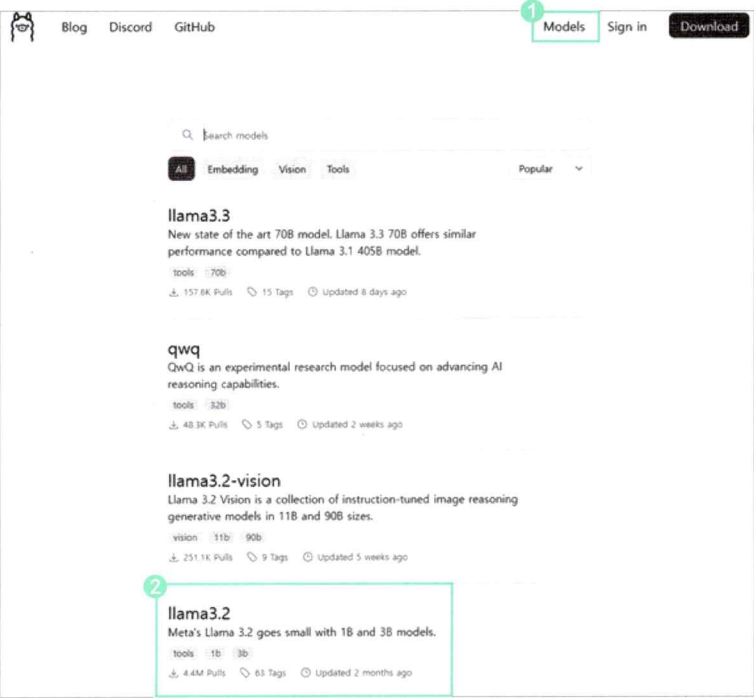
메타는 API로 활용할 수 있는 대규모 언어 모델과 로컬에서 구동할 수 있는 소규모 언어 모델을 모두 제공합니다. 이 책에서는 로컬에 설치하여 사용할 수 있는 소규모 언어 모델의 구동법을 알아보겠습니다.

◆ 라마 API 사용법은 공식 문서(<https://docs.llama-api.com/quickstart>)에서 확인하세요.

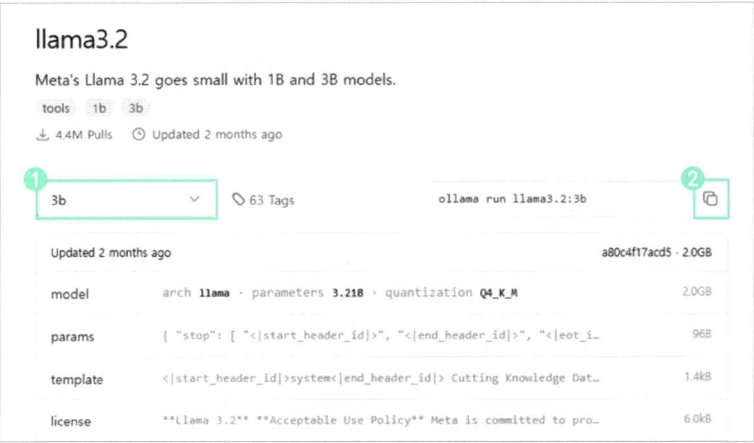
메타의 라마 언어 모델을 로컬 환경에 설치하는 방법은 여러 가지 있습니다. 허깅페이스에서 모델을 직접 내려받거나 메타에서 제공하는 올라마(Ollama)를 통해 내려받을 수 있습니다. 여기에서는 올라마를 활용해 설치하겠습니다.

◆ 올라마를 아직 설치하지 않았다면 11-1절을 참고하세요.

1. 올라마 웹 사이트(<https://ollama.com>)에 접속해 상단의 [Models] 버튼을 클릭하고 사용할 모델을 선택합니다. 최신 모델은 llama 3.3인데, 이 모델은 43GB의 저장 공간과 24GB 이상의 메모리, 최신 그래픽 카드가 필요하므로 이번 실습에서는 llama 3.2의 3b 모델을 사용하겠습니다. 여기서 3b는 이 모델의 매개변수를 의미합니다. 같은 모델이라도 매개변수가 많을수록 성능이 좋고 계산 자원이 많이 필요합니다. llama 3.2의 3b 모델은 2GB로 용량이 비교적 작아서 개인 컴퓨터에서도 구동할 수 있습니다.



2. 매개변수는 [3b]로 선택하고  아이콘을 클릭해 설치 명령어를 복사합니다.



3. 복사한 명령어를 VS Code의 터미널 창에 붙여 넣고 실행합니다. 설치가 완료되면 >>>로 시작하는 프롬프트가 나타납니다.

```
(venv) > ollama run llama3.2:3b — 입력
pulling manifest
verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
```

이제 라마와 터미널 창에서 대화할 수 있습니다. 이 모델은 영어에서는 유창하게 작동하지만 한국어는 일본어, 태국어, 중국어가 섞인 형태로 나오기도 하고 질문에 잘못된 대답을 하는 환각 현상이 나타나는 등 일부 한계가 있습니다. 당분간은 영어를 기반으로 하는 프로그램에 사용하는 것이 좋겠네요. 하지만 시간이 흐름에 따라 라마 모델의 한국어 실력도 유창해질 테니 사용 방법을 익혀 두면 좋겠죠?

```
>>> Hello. Nice to meet you. I'm SungYong Lee.
Nice to meet you too, SungYong Lee! I'm an AI designed to assist and chat with users like you. I don't have a personal name, but I'm here to help answer any questions or topics you'd like to discuss. How's your day going so far?
(... 생략 ...)
>>> OK. Can you tell me about JYP and HYBE?
You're likely referring to two major entertainment companies in the K-pop industry.
(... 생략 ...)
Both JYP and HYBE are major players in the K-pop industry, and they've played a significant role in shaping the global popularity of Korean pop music. Which one would you like to know more about?

>>> 나 영어 못하니까 한국어로 설명해줘.
한국의 엔터테인먼트 회사입니다.

JYP 엔터테인먼트는 1997년 박진영によって 설립되어 있습니다. 이 회사는 많은 성공적인 K-pop 그룹을 발견하고 관리하는 데 bekannt합니다. 예를 들어, 위원 girl스, 2PM, 2AM, Day6, ITZY, TWICE와 같은 그룹들이 있습니다.
(... 생략 ...)
JYP 엔터테인먼트와 HYBE는 한국 non 음악의 전통적인 산업이며, 글로벌 인기를 구축하는 데 큰 역할을 했습니다. 어떤 회사에 더 많이 알고 싶은가?
```

4. 테스트가 끝나면 '/bye'라고 입력하거나 **[Ctrl] + [D]**를 눌러 대화를 종료합니다.

```
>>> /bye
(venv) >
```

Do it! 실습 라마 기반으로 간단한 챗봇 만들기

■ 결과 파일: sec01/llama_simple_chatbot.py

랭체인을 사용하면 언어 모델을 간단히 교체할 수 있습니다. 랭체인을 이용해 라마 기반으로 간단한 챗봇을 만들어 보겠습니다.

1. 먼저 라마를 챗봇에 활용하기 위해 패키지를 설치합니다. VS Code의 터미널 창에 다음 명령어를 입력합니다.

◆ 랭체인에서 제공하는 ChatOllama를 자세히 설명한 문서는 <https://python.langchain.com/docs/integrations/chat/ollama/> 링크를 참고하세요.

```
venv > pip install langchain-ollama
```

2. 랭체인을 이용해 간단한 챗봇을 만듭니다. 08-1절에서 챗봇을 만들 때 사용했던 코드 (langchain_multiturn.py)에서 llm을 ChatOllama로 변경합니다. 그리고 ChatOllama에 사용할 모델명을 입력하면 설정이 완료됩니다. 여기서는 llama3.2:3b 모델을 사용합니다.

◆ 자신의 컴퓨터 사양과 이 책을 읽는 시점에 맞춰 적절한 모델을 선택하세요.

라마로 구현한 간단한 챗봇

■ llama_simple_chatbot.py

```
# from langchain_openai import ChatOpenAI
from langchain_ollama import ChatOllama
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

# 모델 초기화
# llm = ChatOpenAI(model="gpt-4o-mini")
llm = ChatOllama(model="llama3.2:3b")

messages = [
    SystemMessage("You are a helpful assistant."),
]
```

```

while True:
    user_input = input("You\t: ").strip()

    if user_input in ["exit", "quit", "q"]:
        print("Goodbye!")
        break

    messages.append(HumanMessage(user_input))

    response = llm.invoke(messages)
    print("Bot\t: ", response.content)

    messages.append(AIMessage(response.content))

```

이 코드를 실행하고 영어로 질문하면 챗봇이 잘 작동합니다.

사용자: Hello. How are you?

AI: I'm just a computer program, so I don't have feelings like humans do, but thank you for asking! I'm here to help answer any questions or concerns you may have, so please feel free to share what's on your mind. Is there something specific you'd like to talk about or ask?

사용자: What is the biggest problem of Korea do you think?

AI: Korea, as a country, faces various challenges, and it's difficult to identify a single biggest problem. However, some of the key issues that are widely discussed include:

1. **Aging population**: South Korea has one of the highest aging populations in the world, with over 14% of its population above 65 years old. This puts pressure on the pension system, healthcare services, and social security.

(... 생략 ...)

However, if I had to identify one area that's particularly pressing, it would be **the aging population**. The rapid aging of the Korean population has significant implications for the country's social security systems, pension funds, and overall economic stability.

What do you think? Would you like me to elaborate on any of these issues or discuss other aspects of Korea's challenges?

다만 llama 3.2:3b 모델에서는 아직 한국어를 깔끔하게 처리하지 못합니다.

You: 안녕?
 Bot: 안녕하세요! (Hello!) How can I assist you today?
 You: 한국어로만 대답해
 Bot: 안녕하세요! 한국에서 어떤 `help`이 필요하신가요?
 You: `help`이 어느 나라 말이야 무슨 뜻이야?
 Bot: "`help`"는 아랍어와 페르시아어의 영향을 받은 한국어로 "help"나 "assist"를 의미합니다. 일반적으로 "`help`"라고 발음하여 "`도움`"과 같은 의미를 가지고 있습니다.
 You:

곧 라마의 소규모 언어 모델도 한국어를 잘하게 될 것이고 허깅페이스에서 받을 수 있는 다른 모델의 성능도 크게 향상될 것입니다. 시기에 따라 발전된 모델을 선택해서 사용하길 바랍니다.

Do it! 실습 로컬 임베딩 모델 사용하기

■ 결과 파일: sec01/embedding_bge_m3.ipynb

09장과 13장에서 RAG를 개발할 때 임베딩 모델로 오픈AI의 `text-embedding-3-large`를 API로 사용했습니다. 그런데 사용할 PDF 파일을 외부로 반출할 수 없거나 인터넷이 연결되지 않는 환경에서도 실행되는 프로그램을 개발한다면 이 방식을 사용할 수 없습니다. 여기서는 로컬 환경에서 인터넷이 연결되지 않아도 실행할 수 있는 프로그램을 만들어 보겠습니다.

로컬에서 사용할 수 있는 언어 모델이 있는 것처럼 로컬에서 사용할 수 있는 임베딩 모델도 여러 가지가 있습니다. 하지만 아직 대부분의 모델이 영어로 학습되어 있어서 한국어 문서에 사용할 때 동일한 성능을 보장하기는 어렵습니다. 여기서는 BAAI/bge-m3 모델을 사용하겠습니다. 이 모델 역시 모든 한국어 문서에서 최적의 성능을 보장하지는 않지만 초보자도 활용하기 쉽고 성능도 괜찮습니다. 여러분이 이 책으로 공부하는 시점에는 성능이 더 좋은 모델이 출시되었을 수 있습니다. 몇 가지 모델을 골라 사용해 보고 문서별로 가장 적합한 모델을 사용하길 바랍니다.

1. 새로운 주피터 노트북 파일 `embedding_bge_m3.ipynb`와 `data` 폴더를 만들고 사용할 PDF 파일들을 넣습니다. 09장에서 사용한 도시 계획 문서들을 활용해 임베딩해 보겠습니다. 여러분이 원하는 PDF 문서를 활용해도 좋습니다.

PDF 문서 준비하기

embedding_bge_m3.ipynb (1)

```
from glob import glob
```

```
for g in glob('../data/*.pdf'):
    print(g)
```

이 셀을 실행하면 다음과 같이 ./data 폴더 내의 PDF 파일들이 출력됩니다.

```
../data\2040_seoul_plan.pdf
../data\OneNYC_2050_Strategic_Plan.pdf
```

2. 13장에서 만든 파일(rag_with_langgraph.ipynb)에서 PDF 파일 경로를 입력하면 청크 사이
즈가 1000, 오버랩이 100이 되도록 분할하는 코드를 복사해 셀에 붙여 넣습니다.

 청크 분할하기

 embedding_bge_m3.ipynb (2)

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

def read_pdf_and_split_text(pdf_path, chunk_size=1000, chunk_overlap=100):
    """
    주어진 PDF 파일을 읽고 텍스트를 분할합니다.
    매개변수:
        pdf_path (str): PDF 파일의 경로.
        chunk_size (int, 선택적): 각 텍스트 청크의 크기. 기본값은 1000입니다.
        chunk_overlap (int, 선택적): 청크 간의 중첩 크기. 기본값은 100입니다.
    반환값:
        list: 분할된 텍스트 청크의 리스트.
    """
    print(f"PDF: {pdf_path} -----")

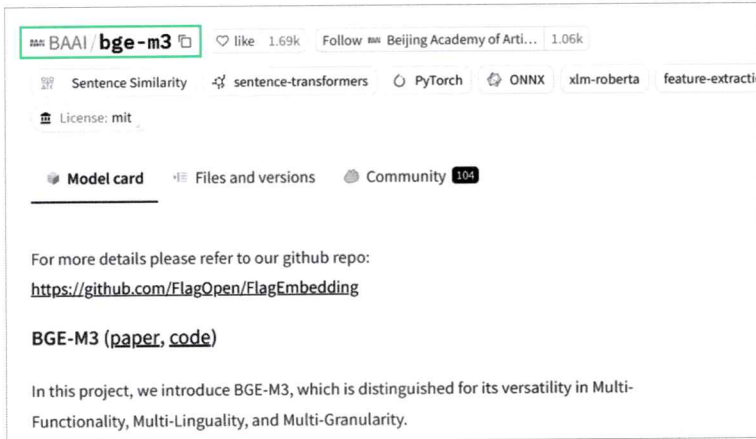
    pdf_loader = PyPDFLoader(pdf_path)
    data_from_pdf = pdf_loader.load()

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size, chunk_overlap=chunk_overlap
    )

    splits = text_splitter.split_documents(data_from_pdf)

    print(f"Number of splits: {len(splits)}\n")
    return splits
```


3. 이제 임베딩 모델을 설치하겠습니다. BAAI/bge-m3 모델을 허깅페이스에서 내려받습니다(<https://huggingface.co/BAAI/bge-m3>). 이 모델은 중국에서 개발한 임베딩 모델로 다국어를 지원합니다.



4. 허깅페이스에서 모델을 쉽게 내려받기 위해 다음 명령어를 입력해 langchain_huggingface를 설치합니다.

langchain_huggingface 설치하기
embedding_bge_m3.ipynb (3)

```
%pip install langchain_huggingface
```

5. 만약 CUDA를 지원하는 NVIDIA 그래픽 카드를 갖고 있다면 파이토치를 설치합니다. GPU를 활용해 빠르게 임베딩할 수 있어서 유용합니다. 파이토치 웹 사이트에서 환경에 맞는 명령어를 복사해 설치하세요(<https://pytorch.org/get-started/locally/>). ◆ 파이토치 설치: 05-2절 '위스퍼 모델을 내려받아 로컬에서 사용하기' 실습을 참고하세요.

파이토치 설치하기
embedding_bge_m3.ipynb (4)

```
%pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu124
```

6. 임베딩 모델을 선언합니다. 저는 `device`를 `cuda`로 설정했지만 CUDA를 지원하지 않는 컴퓨터라면 이 부분에 `cpu`라고 입력합니다. `normalize_embeddings`를 `True`로 설정하면 임베딩한 벡터값을 정규화해 줍니다.

임베딩 모델 선언하기 embedding_bge_m3.ipynb (5)

```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="BAAI/bge-m3",
    model_kwargs = {'device': 'cuda'},          # cuda 지원하면 cuda, 아닌 경우 cpu
    encode_kwargs = {'normalize_embeddings': True}, # 임베딩 정규화
)
```

7. 임베딩 모델이 잘 작동하는지 확인하기 위해 다음처럼 입력해 실행합니다.

임베딩 모델 테스트하기 embedding_bge_m3.ipynb (6)

```
embeddings.embed_documents("안녕하세요")
```

다음처럼 벡터로 잘 변환되면 성공입니다.

```
[[0.0024569211527705193,
 0.03226257115602493,
 -0.007424186449497938,
 0.005268432199954987,
 ...
 (... 생략 ...)]
```

8. 준비한 PDF 파일을 읽어 텍스트를 임베딩하겠습니다. 다음은 09장과 13장에서 사용했던 코드와 동일합니다. 현재 작업 중인 폴더에 `chroma_store` 폴더가 없다면 새 크로마 DB를 생성합니다. `Chroma.documents`의 `embeddings`을 이전 셀에서 `bge-m3`로 선언했으므로 해당 모델로 임베딩됩니다. `chroma_store` 폴더가 이미 존재하면 임베딩을 완료한 상태라는 의미이므로 새로 임베딩하지 않습니다. 만약 앞에서 실습한 오픈AI의 임베딩 모델로 생성된 크로마 DB가 남아 있다면 오류가 발생하므로 기존 DB를 삭제하고 이 셀을 실행하세요.

```

from langchain_chroma import Chroma

import os
persist_directory='../chroma_store'

if os.path.exists(persist_directory):
    print("Loading existing Chroma store")
    vectorstore = Chroma(
        persist_directory=persist_directory,
        embedding_function=embeddings
    )
else:
    print("Creating new Chroma store")

    all_splits = []
    for g in glob('../data/*.pdf'):
        all_splits.extend(read_pdf_and_split_text(g))

    print(f"Total number of splits: {len(all_splits)}")

    vectorstore = Chroma.from_documents(
        documents=all_splits,
        embedding=embeddings,
        persist_directory=persist_directory
    )

```

9. 청크를 잘 가져오는지 리트리버를 테스트해 봅시다. 다음 코드는 13-1절에서 사용한 것과 동일합니다.

```

retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

chunks = retriever.invoke("서울시 온실가스 저감 정책")

for chunk in chunks:
    print(chunk.metadata)
    print(chunk.page_content)

```

체크 5개 중에서 2~3개 정도가 관련 있는 내용이었고 다른 도시의 계획이 나오기도 했습니다. 서울시 도시 계획에 관한 쓰레기 저감 정책은 부족한 부분이 있지만 이 정도면 충분히 활용할 수 있는 리트리버가 된 것 같습니다.

```
{'page': 65, 'page_label': '66', 'source': '../data\2040_seoul_plan.pdf'}
```

제3절 2040 서울도시기본계획 7대 목표 57

「특히, 서울시는 현재 온실가스 배출량의 90%를 차지하고 있는 건물과 수송 부문 감축을 위해 적극적인 대책을 마련하고 있다.

(... 생략 ...)

```
{'page': 556, 'page_label': '557', 'source': '../data\2040_busan_plan.pdf'}
```

517

(... 생략 ...)

이제 인터넷 연결 없이 오픈AI에 비용을 지불하지 않고도 임베딩을 할 수 있습니다. 하지만 여러분이 활용해야 하는 자료에 대해 임베딩과 벡터 검색이 잘 되는지 반드시 검토해 보고 적절한 모델을 선택해야 함을 잊지 마세요.

★ 한 걸음 더!

임베딩 모델을 어디에서 찾을 수 있나요?

허깅페이스에 임베딩 모델의 성능을 순위로 매겨 놓은 웹 페이지(<https://huggingface.co/spaces/mteb/leaderboard>)가 있습니다. 이 웹 페이지는 MTEB라는 지표를 바탕으로 순위를 산정한 결과를 보여 줍니다. 여기에서 자신이 주로 사용할 언어와 용도를 결정하면 해당 조건에 맞춰 순위를 보여 줍니다.

MTEB는 Massive Text Embedding Benchmark의 줄임말로 다양한 임베딩 작업에서 텍스트 임베딩 모델의 성능을 평가하는 대규모 벤치마크입니다. MTEB는 8가지 임베딩 작업에 걸쳐 총 58개의 데이터셋과 112개의 언어를 포함하고 있습니다.

그러나 이 지표를 절대적으로 믿을 수는 없으므로 내가 사용하려는 데이터에 실제로 적용해 보고 결과를 확인하는 과정이 필요합니다.



16-2 LLM에 기반한 서비스 발전시키기

빠른 답변 vs 사용자가 원하는 답변

GPT에 기반한 인공지능 프로그램을 개발할 때 답변을 빠르게 제공하는 것이 중요할지 사용자가 원하는 답변을 제공하기 위해 답변 내용 생성 과정을 더 구조화해야 할지 고민하는 경우가 종종 있습니다. 이런 경우 상황을 고려한 개발이 필요합니다.

예를 들어 GPT를 활용해 게임 속 NPC를 만드는 경우를 생각해 봅시다. 게임에서는 2~3초만 반응이 지연되어도 사용자가 어색하게 느낄 수 있습니다. 또한 NPC가 너무 긴 문장을 말하면 부자연스럽게 보일 수 있습니다. 이런 경우에는 반응 속도를 고려해 짧고 즉각적인 답변을 생성하는 것이 더 적합합니다. 반면 책이나 보고서를 작성하는 프로그램처럼 시간이 걸리더라도 GPT가 정확하고 구조화된 답변을 제공해야 할 때도 있습니다. 이럴 때는 랭그래프를 활용해 언어 모델의 출력 방식을 구조화함으로써 고품질의 답변을 생성할 수 있습니다. 이를 통해 언어 모델이 단순히 텍스트를 생성하는 것에 그치지 않고 사용자가 원하는 방식으로 작업하도록 유도해야 합니다. 다만 이 방식은 과정마다 판단이 필요하므로 답변 시간이 길어질 수 있습니다. 이때 사용자가 지루함을 덜 느끼도록 GPT가 현재 어떤 작업을 하고 있는지 표시하거나, 완성된 문장을 스트림 방식으로 출력하거나, 답변이 나올 때까지 로딩 아이콘을 띄우는 등 다양한 방법으로 대기 시간을 관리해야 합니다.

특히 고객의 의뢰를 받아 프로그램을 개발하는 경우 GPT가 어떤 단계에서 작업을 하고 있는지 그 과정을 고객에게 공유하는 것도 중요합니다. GPT가 답변을 만들어 가는 과정을 이해시키고 공감을 얻을 수 있도록 노력해야 합니다.

기능 개발이 끝났으면 그때부터 시작이다

GPT 같은 언어 모델에 기반한 프로그램이나 서비스는 기능 개발을 위한 코딩이 끝났다고 해서 모든 작업이 완료된 것은 아닙니다. 사실 그 시점이 진짜 시작이라고 할 수 있습니다.

사용자는 언어 모델을 사용할 때 항상 정확한 답변을 받을 것이라고 기대합니다. 그러나 GPT를 비롯한 언어 모델은 완벽하지 않으므로 오류를 줄이려면 지속적으로 사용자의 질의와 언어 모델이 생성한 답변을 모니터링해 서비스를 개선하고 조정해야 합니다. 예를 들어 언어 모델의 환각 현상 문제를 해결하기 위해 이 책에서는 RAG 기술을 활용하거나 웹 검색, 유튜브 검색 기능을 추가하는 방법, 랭그래프를 사용해 답변을 검토하는 방법을 소개했습니다. 이러

한 기술들을 적절히 조합함으로써 더 나은 성능을 발휘하는 AI 에이전트로 발전시킬 수 있습니다. 또한 항상 최신 기술을 적극적으로 학습하고 적용하려고 노력해야 합니다.

언어 모델에 기반한 프로그램을 개발할 때에는 사용자와 반드시 소통해야 합니다. 언어 모델에서는 오류를 완전히 없애기 어려우므로 이 부분을 사용자에게 미리 안내하고 공감을 얻어야 합니다. 또한 문제가 발생했을 때 개발자와 사용자가 함께 해결 방안을 찾아 나가는 과정이 중요합니다. 이를 통해 언어 모델의 한계를 극복하고 더 나은 성능과 신뢰를 갖춘 결과물로 발전시킬 수 있습니다.

기존의 프로그래밍은 명확한 코드 규칙에 따라 설계되므로 기능을 모두 개발하면 완성된 것으로 여겼습니다. 하지만 언어 모델에 기반한 프로그램이나 서비스는 다릅니다. 기능 개발이 완료된 시점은 단지 시작에 불과하며 수많은 테스트와 피드백을 거쳐 개선해 나가야 함을 잊지 말아야 합니다.